



The workspace: both repos, their remotes, and the rules that span them

This file was [CLAUDE.md](#) at the folder root until 2026-09-06. It moved into the repo when Syncthing was retired, because a file outside a pushed repo now reaches exactly one machine. The folder root keeps a short pointer to it.

Personal project. Fork of [YARG](#) (Yet Another Rhythm Game), plus a new self-hosted song server. Everything here is LGPL-3.0-or-later, matching upstream.

Goal

Further develop YARG: graphics, controller compatibility (prioritising official Rock Band and Guitar Hero instruments), and major backend functionality. Contributions should stay upstreamable where possible.

#1 priority: a Docker-compatible song server for the YARG client, portable to Raspberry Pi, macOS and Windows. Once server/client works, additional features are modular and enabled from a config menu in the server app.

Repos and where they push

Two repos live under this folder. Both are personal, so they follow the personal chain.

Folder	GitLab (origin, source of truth)	Public mirror
yarg-song-server/	gitlab.badassium.com/fatalexception/yarg-song-server (project 53)	github.com/Coffeehedake/yarg-song-server (GitLab mirror 10)
yarg/	gitlab.badassium.com/fatalexception/yarg (project 55)	github.com/Coffeehedake/yarg (GitLab mirror 11)

Both mirrors are one-way, all branches, divergent refs not kept, and were verified end to end on 2026-09-05. [Coffeehedake/yarg](#) is a real GitHub **fork** of [YARC-Official/YARG](#) — the only shape GitHub accepts an upstream pull request from.

Rules:

- **Push to [origin](#) (Vault2 GitLab) only.** GitHub is a downstream, force-overwritten mirror — never push to it directly, never commit to it, never open the PR there first.
- **LFS does propagate.** An earlier note here claimed GitLab push mirroring drops LFS objects. That was wrong and has been measured: on GitLab 18.11.3, a fresh 1 MB LFS object pushed to a non-fork GitLab project arrived in GitHub's own LFS storage and downloaded with a matching SHA-256. Nothing special is needed when client work adds a [.png](#), [.jpg](#), [.exr](#), [.fbx](#) or [.ttf](#) — the five patterns YARG's [.gitattributes](#) tracks.
- **The mirror force-overwrites, and the GitHub side is a fork.** Anything done directly on GitHub — a branch pushed there, a merge into the fork's [dev](#) — is clobbered on the next sync. To open an upstream PR: create the branch on GitLab, let it mirror, then open the PR from the mirrored branch, and do not touch it on GitHub afterwards.
- [yarg/](#) additionally carries an [upstream](#) remote pointing at [YARC-Official/YARG](#). **Upstream PRs target [dev](#), never [master](#)** — upstream will refuse [master](#) PRs outright.
- [yarg/](#) **is not cloned locally yet, on purpose.** The fork is fully imported on Vault2 (21 branches, ~350 MB with LFS) with its default branch set to [dev](#), and nothing touches the client until Phase 3. The clone command is in [yarg-song-server/docs/ROADMAP.md](#) under Phase 0.
- **You do not need a GitHub credential for either repo, and should not go looking for one.** GitLab owns the mirror credential and pushes for us. Measured 2026-09-06: mirror 10 on project 53 last succeeded at [20:42:42](#), the same minute as that push to origin, with an empty [last_error](#); mirror 11 on project 55 is enabled and last succeeded 2026-09-05. A GitHub PAT is only needed for something GitLab cannot do on our behalf — opening a pull request against [YARC-Official](#) through the API, say.
- **A dead mirror is silent, and this is where it shows.** Nothing alerts if the mirror's stored credential expires: pushes to origin keep succeeding, and GitHub simply stops moving. The only signal is the API, so check it rather than assuming the mirror ran — same failure shape as the Unraid disk alerts that have not fired since March 2025.

```
powershell $tok = (& pwsh -NoProfile -File 'C:\dev\_environment\Get-DevCredential.ps1'
-Target 'dev:gitlab-ce-pat' | Out-String).Trim() foreach ($proj in @(53,55)) { (Invoke-
RestMethod "https://gitlab.badassium.com/api/v4/projects/$proj/remote_mirrors" ` -
Headers @{ 'PRIVATE-TOKEN' = $tok }) | ForEach-Object { "project ${proj}:"
enabled=${($_.enabled)} status=${($_.update_status)}
last_success=${($_.last_successful_update_at)} err='${($_.last_error)}' } }
```

Note [\\$proj](#), not [\\$pid](#) — **\$PID is a read-only automatic variable in PowerShell** and assigning to it fails the whole loop with an error that points at the [foreach](#), not at the name. Same family as [r](#) ([Invoke-History](#)) and [h](#) ([Get-History](#)). - Vault2 GitLab-CE API token:

`op://fallout-automation/26tovxp2kthsdekw7dql4zeqzy/credential` , or `dev:gitlab-ce-pat` in Windows Credential Manager — **prefer Credential Manager for anything polled**, since `op` reads are rate-limited per service account. Enable the headless 1Password service account first: `C:\dev\environment\enable-headless-op.ps1` . Never put a token in chat, in a commit, or in a file. - **GitHub PAT, when one is genuinely needed:**

`op://MCP/xzslly5bgd2orqj2xy524txnfui/credential` — vault `MCP` , item titled exactly `GitHub PAT` . Same item the credentials list already calls "Coffeedake GitHub".

The trap is in `Home-Lab` , not in the name. That vault holds `GitHub PAT (pc-deploy hipot-autobuild read-only)` — **read-only, and belonging to another project** — whose title is the closest match to a bare "GitHub PAT" search. Grabbing it gets a token that authenticates fine and then fails on the first write, which reads as a permissions bug in whatever you were doing rather than as the wrong token. `Home-Lab` also holds `GitHub MCP PAT (Claude Desktop, ENG-1)` , which is a third, separate credential for the Claude Desktop MCP server.

So: **address it by ID in vault `MCP`** , and do not search across vaults by title. Note that `op item list` with no `--vault` rate-limits the whole service account for the better part of an hour, so a title search is expensive as well as ambiguous.

Stack decisions

- **Server: Go.** Single static binary, trivial cross-compile to arm64 (Pi), macOS and Windows, tiny container. See `yarg-song-server/docs/ADR-001-server-architecture.md` .
- Go means **YARG.Core is not available** — the server reimplements the parts of the song format it needs. **Before writing any parser code, read `yarg-song-server/docs/SOURCES.md`** : much of this is documented on the official wiki and by TheNathannator, and deriving `song.ini` from source when a better spec existed cost this project four real defects.
- **The server is a content source, not a game feature.** It emits ordinary `.sng` files that an unmodified YARG can already read. Client-side integration comes later and separately.

Hard constraints

- **Never implement CON / mogg decryption.** Upstream's `CONTRIBUTING.md` puts "CON Decryption" in the Out of Scope tier ("your PR will immediately be denied"), and it carries real DMCA 1201 exposure. The server refuses `.con` / `_rb3con` / `.pkg` on ingest with an explicit message.
- **Never generate `songcache.bin`** . Not because it is unreadable — it is a plain binary file and an external tool could write one. Because it stores **absolute local paths**, so a cache built on a server is meaningless to a client; because `CACHE_VERSION` is a date stamp checked with no compatibility window and no migration, over a field layout that is `internal` with no version of its own; and because getting it wrong fails *silently* — YARG just rebuilds and the tool appears to have worked. The server ships its own JSON catalog instead.

- **Never distribute copyrighted audio.** The eventual auto-charting feature must be able to emit chart/vocal tracks as a package separate from the music, so charts can be shared without audio.

Conventions

- Docs are written in the same change as the code, never batched for later.
- Branded PDFs use the **FatalException** brand (personal project) — never Juniper.
- Verify the local repo is actually up to date against origin before starting work AND again before committing; the other machine pushes mid-session.
- **THE TWO MACHINES ARE INDEPENDENT. Nothing syncs between them.** Syncthing was retired on 2026-09-06 and nothing replaced it: on ENG-1 the logon task is disabled, no process runs and nothing listens on 8384/22000; the Vault2 container sits in `Created` and has never started. So `git push` to origin is the only thing that moves work between ENG-1 and r7 — an uncommitted file, or a file in no repo at all, exists on exactly one machine.

Two consequences that bite immediately:

- **This file is inside the `yarg-song-server` repo for that reason.** It used to live at the folder root, outside any repo, on the assumption that the folder itself synced. The moment that assumption died the file reached nowhere. Anything a future session must read has to be *in a repo that is pushed* — the folder is not a transport.
- The Cowork project *card* does not sync either, and never did. Recreate it per machine and point it at this folder. These instructions live here precisely so nothing is trapped in the card.

Branding

FatalException, not Juniper — this is a personal GitLab project. Render docs with the central builder, never a project-local copy:

```
python "C:\dev\fatalexception-brand-kit\scripts\build-pdfs.py" "C:\dev\YARG - Open Source Contributions\yarg-song-server"
```

Regenerate the affected PDF in the same change as the markdown, so the two never drift.

Tooling installed for this project

All per-user under `%LOCALAPPDATA%\Programs\`, no elevation, each deletable as one folder:

What	Where	Why
YARG v0.15.0	<code>Programs\YARG\YARG.exe</code>	The oracle. The only thing that can say whether a package we produced is really acceptable
SngCli v0.3.0	<code>Programs\sngcli\win-x64\SngCli.exe</code>	The reference <code>.sng</code> encoder/decoder
Go 1.27.0, mingw-w64 GCC 16.2.0	<code>Programs\go</code> , <code>Programs\mingw64</code>	Toolchain; the GCC is what makes <code>go test -race</code> work at all

Running the oracle — worth knowing, because it has found bugs no unit test did:

```
go run ./cmd/mkcorpus -out $env:USERPROFILE\yarg-test\corpus
# point YARG at it: edit SongFolders in
#   %USERPROFILE%\AppData\LocalLow\YARG\YARG\release\settings.json
# delete songcache.bin beside it, launch YARG, wait ~45s, then read badsongs.txt
```

`badsongs.txt` is YARG's own verdict on every song it refused, and the song cache's readable strings show which titles it accepted. That loop found three real bugs our tests had passed.

Gotchas

The folder name has spaces

`YARG - Open Source Contributions` contains `-`. Any tool invoked with an unquoted path splits it into separate arguments — this has broken the PDF builder twice, resolving a lone `-` against the home directory and reporting a confusing `C:\Users\ENG2\-` not-found that reads like the ENG-1 wrong-machine-path quirk but is not. **Always quote the path**, including inside `Start-Process -ArgumentList`, which does no quoting of its own.

Never give a PowerShell helper a single-letter name

The alias wins. `r` is `Invoke-History` and `h` is `Get-History`, so a `function R` fails every call with *"Cannot locate the history for command line ..."* and a `function H` with *"Cannot bind parameter 'Id'"*. Both happened here, days apart, and both read as a bug in the thing being called. Assume every letter is taken; name helpers `Get-SngHashes`, not `H`.

`$_` and `$env:` are mangled inside an inline `pwsh -Command`

The outer shell expands them before `pwsh` ever sees the string, so a pipeline using `$_` becomes a parser error about a missing operand. **Write a `.ps1` and run `pwsh -NoProfile -File`**. This is not an occasional annoyance — every inline command with a `ForEach-Object` block fails this

way.

PowerShell variable names are case-insensitive

`$C` and `$c` are the same variable. A `foreach ($c in $cases)` loop silently overwrote a `$C` holding the corpus path, and the resulting failure looked exactly like a bug in the Go code — `open .: The system cannot find the path specified` — for two tool calls. Use distinct, wordy variable names in any loop.

A long child process dies with the bridge

A process started from a Cowork session is killed when a bridge call times out, which silently cancelled a `winget` install mid-download. For anything running longer than about a minute, register a **scheduled task**; it is detached and survives.